GROUP 4 · FINAL SUBMISSION

Energy Forecasting Pipeline

An End-to-End Time Series Pipeline for PJM Hourly Electricity Demand

EDA & Modeling · MySQL + MongoDB · FastAPI CRUD · Forecast Script

Member	Task Owned
Hassan Adelani Luqman	Task 1 — Time-Series EDA, Preprocessing & Modeling
Abdalaziz Twariki	Task 2 — Database Design (SQL & MongoDB)
Nkhomotabo Amazing Mkhonta	Task 3 — CRUD & Time-Series API Endpoints
Parfait Shema Semana	Task 4 — Prediction / Forecast Script

GitHub Repository:

`https://github.com/Hassan-Adelani-Luqman/group4-energy-forcasting-pipeline`

Table of Contents

1. Problem Definition & Dataset Justification

2. Task 1 — Time-Series EDA, Preprocessing & Modeling

3. Task 2 — Database Design (SQL & MongoDB)

4. Task 3 — CRUD & Time-Series API Endpoints

5. Task 4 — Prediction / Forecast Script

6. Team Contributions

7. Limitations & Future Work

1. Problem Definition & Dataset Justification

Problem. Given a region's historical hourly electricity demand (MW) plus calendar context (hour, day of week, month, US federal holidays), predict near-term future demand for that same region. This is a univariate time series forecasting problem with exogenous calendar features — the target depends heavily on its own recent past (intra-day and intra-week cycles) rather than on unrelated external signals.

Why this dataset. The PJM Hourly Energy Consumption dataset is real operational load data from a US grid operator (PJM Interconnection), not a synthetic series — short-horizon demand forecasting of exactly this kind is used industrially for generation dispatch and grid balancing, giving the project direct real-world relevance. Two regions were used rather than one:

- **PJME** and **AEP** are service areas of meaningfully different scale (PJME's absolute MW is roughly double AEP's), letting us check whether the same modeling approach generalizes across regions of different size rather than overfitting conclusions to a single area.
- Multiple regions map naturally onto a multi-entity database design (a `regions` table with one-to-many `energy_readings`), so the same processed data serves both the modeling task (Task 1) and the database-loading task (Task 2).
- Each series spans 14+ years at hourly granularity (PJME: 2002–2018, AEP: 2004–2018) — long enough to expose the daily/weekly/yearly cycles that lag and rolling-window features are built to capture.

2. Task 1 — Time-Series EDA, Preprocessing & Modeling Hassan Adelani Luqman

2.1 Dataset Understanding

Region	Time Range	Total Hourly Rows	Missing Hours (before fill)
PJME	2002-01-01 01:00 → 2018-08-03 00:00	145,392	30 (0.0206%)
AEP	2004-10-01 01:00 → 2018-08-03 00:00	121,296	27 (0.0223%)

Both series are hourly once reindexed to a full `DatetimeIndex`; the small number of missing hours (well under 0.1%) come from short gaps such as DST transitions or brief meter outages, not systematic collection issues. **Missing-value strategy:** gaps of 3 hours or less are linearly interpolated (short enough to trust); any longer gaps fall back to forward/backward fill rather than trusting a long linear interpolation across a data-collection outage. PJME's demand operates at a noticeably larger absolute MW scale than AEP, reflecting a larger service area, and both distributions are right-skewed — a baseline load most of the time, punctuated by higher-demand peak periods.

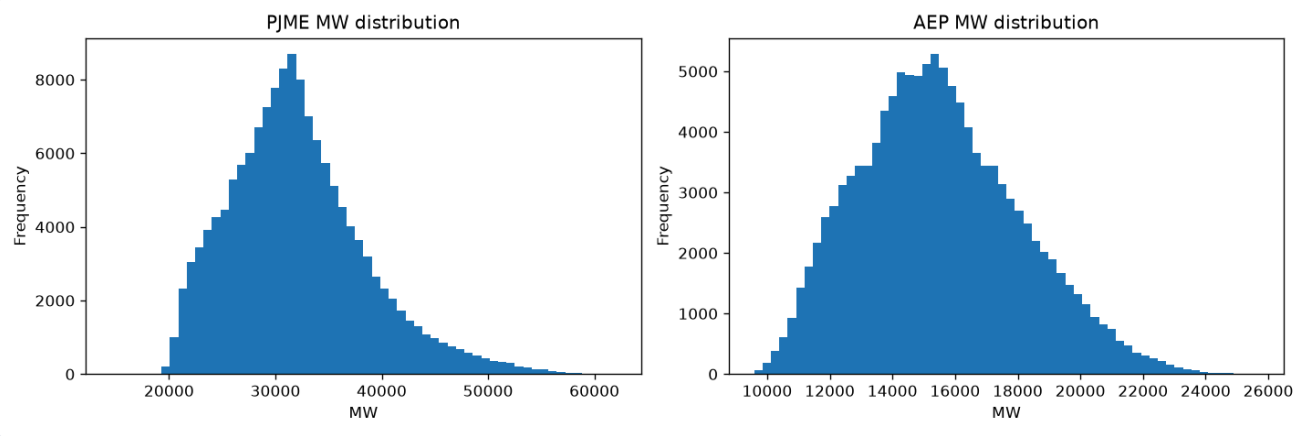


Figure 2.1 — Hourly MW distribution per region: unimodal, right-skewed, PJME at a larger scale than AEP.

2.2 Analytical Questions

Q1. Is there a long-term trend in demand? (STL decomposition)



Figure 2.2 — STL trend component of the daily-mean series, both regions.

The STL trend component does not show a strong monotonic drift for either region over 2002/2004–2018 — demand oscillates within a fairly stable band rather than steadily growing or shrinking. A model leaning on lag/seasonal structure should generalize well without needing an explicit trend term.

Q2. What monthly/yearly seasonality exists?

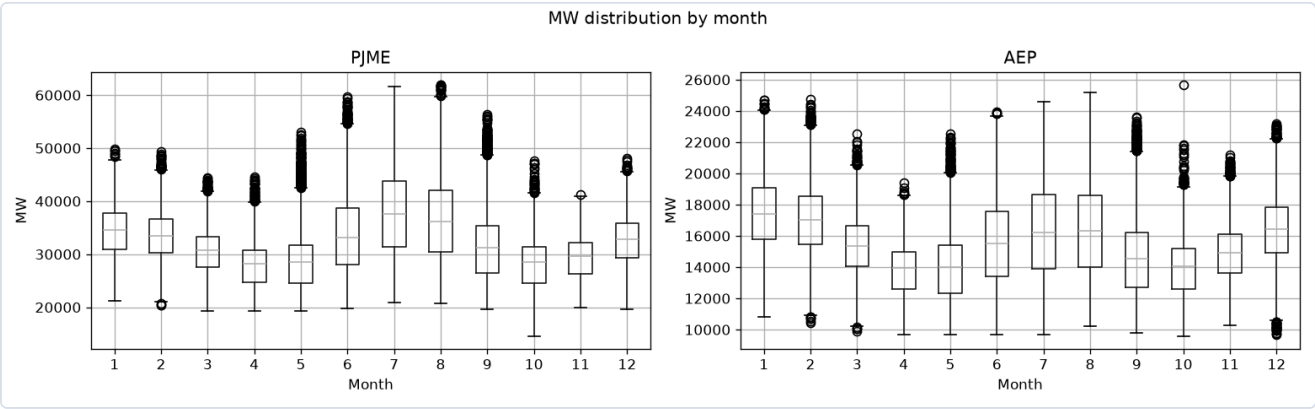


Figure 2.3 — MW distribution by month, both regions.

Both regions show elevated demand and wider spread in summer (cooling load) and winter (heating load) months versus the shoulder seasons — a textbook signature of weather-driven electricity demand, justifying `month` as a calendar feature.

Q3. Is there a weekday-vs-weekend and holiday effect?

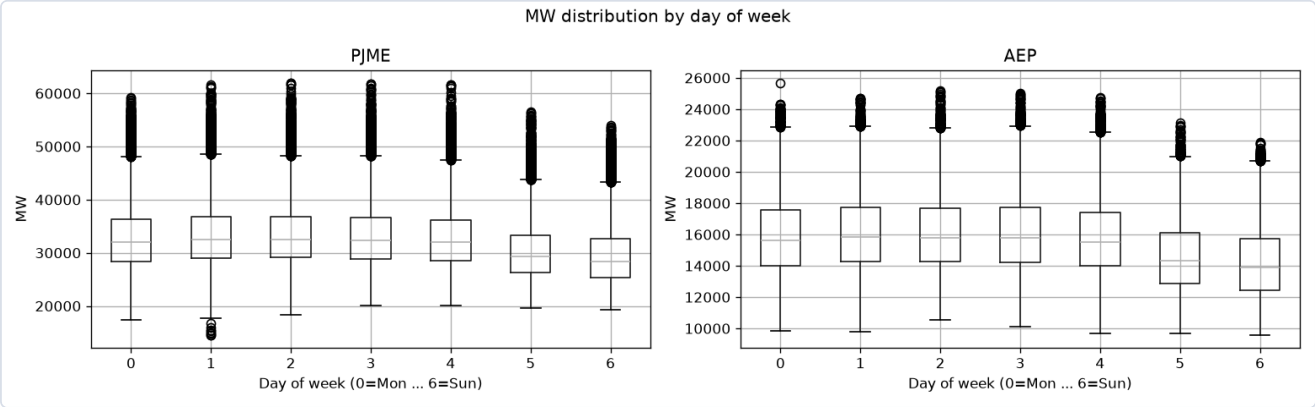


Figure 2.4 — MW distribution by day of week, both regions.

Weekdays show higher median demand than weekends in both regions, consistent with commercial/industrial activity dropping on Saturday/Sunday. US federal holidays show lower average MW than non-holidays, confirming demand is driven substantially by business activity — both `is_weekend` and `is_holiday` are kept as model features.

Q4. [Lag] How strongly does demand at hour t correlate with t-24h and t-168h?

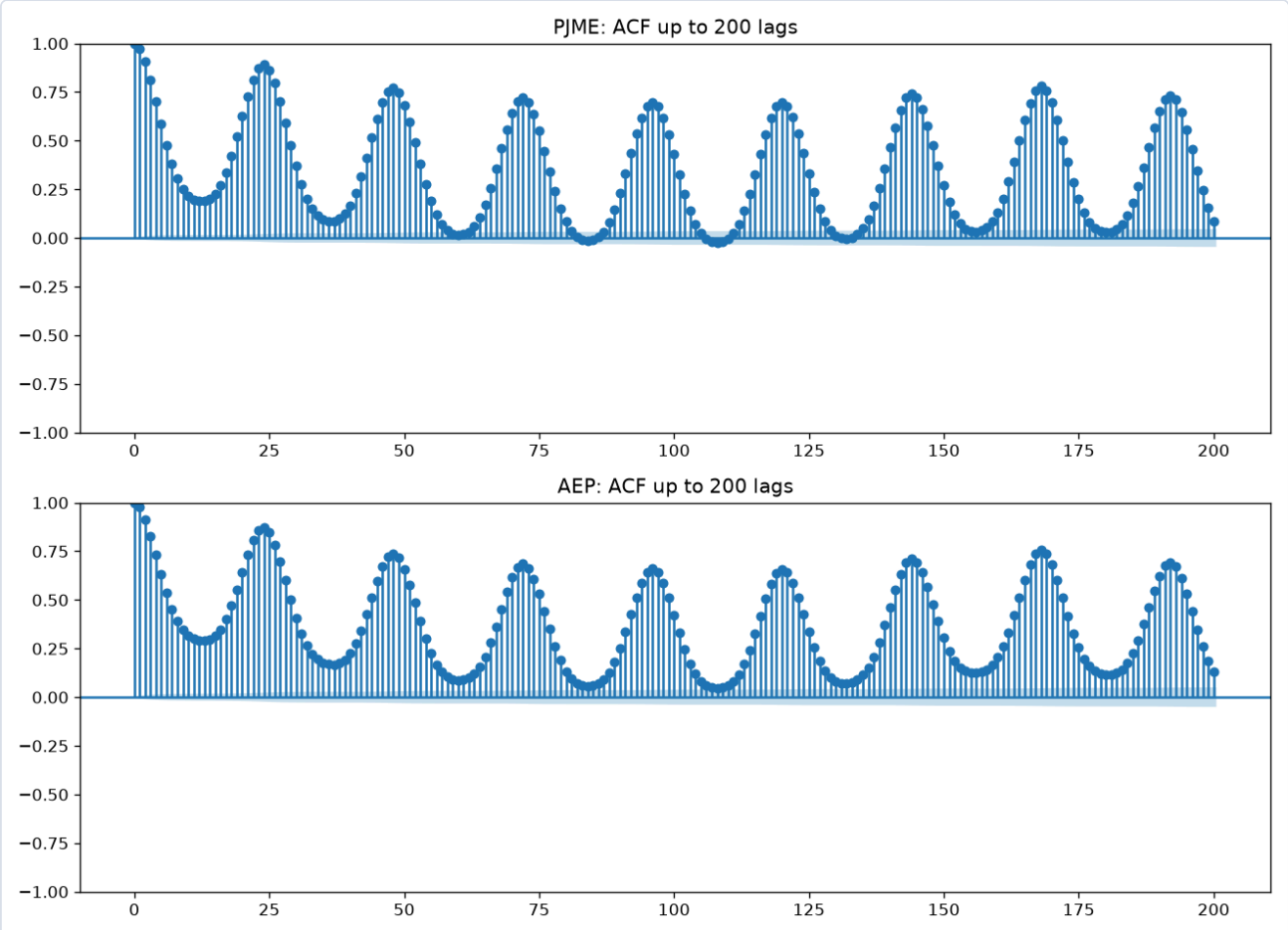


Figure 2.5 — ACF up to 200 lags, both regions.

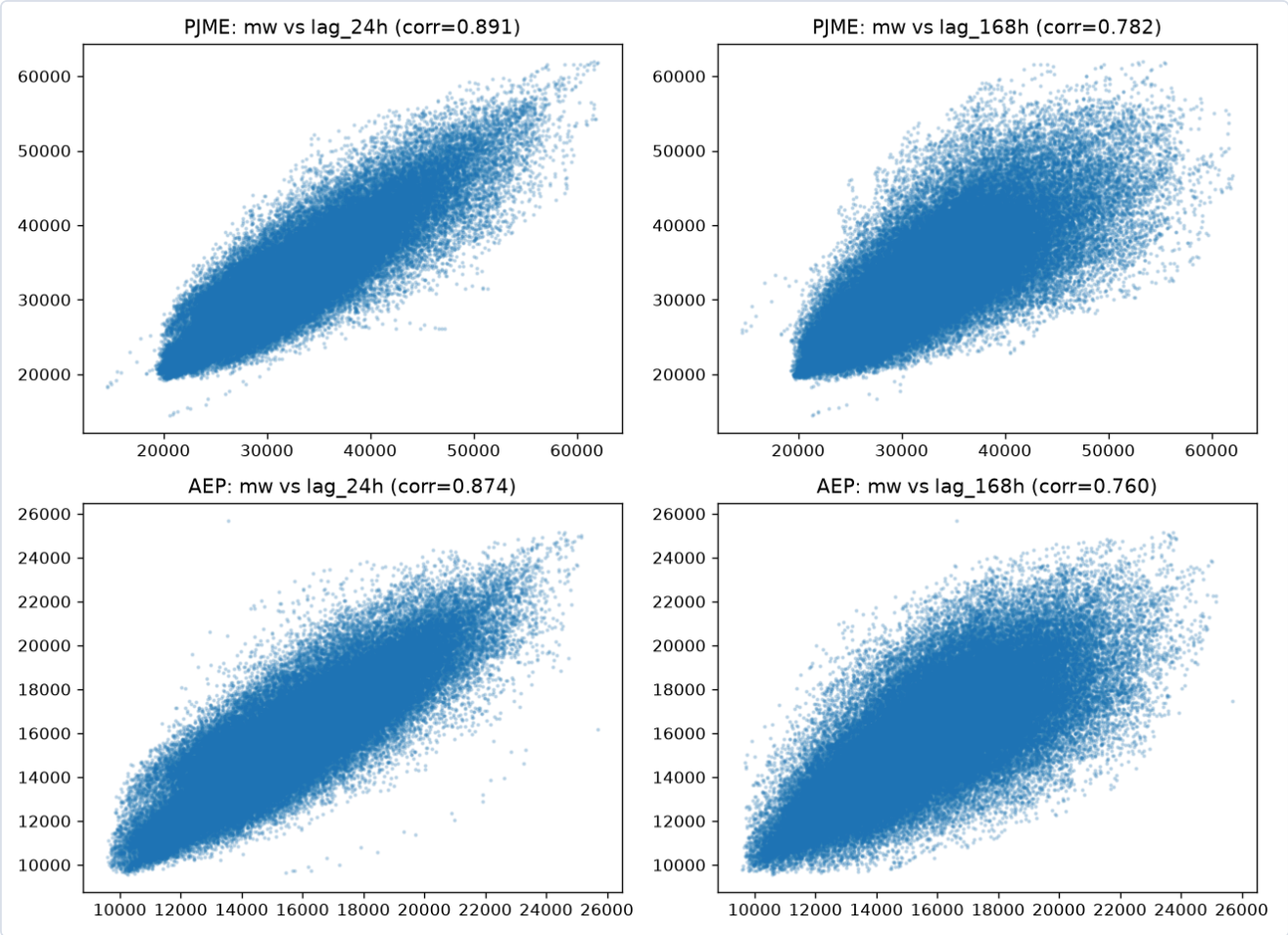


Figure 2.6 — mw vs lag_24h and mw vs lag_168h scatter plots.

The ACF shows strong, slowly-decaying autocorrelation with clear spikes at daily (24h) and weekly (168h) multiples — a hallmark of load data with both daily and weekly cycles. The lag scatterplots confirm this quantitatively: `lag_24h` correlates very strongly with the current value, and `lag_168h` is also strongly correlated, justifying both as lag features.

Q5. [Moving average] How do 24h and 7-day rolling means expose regime shifts?

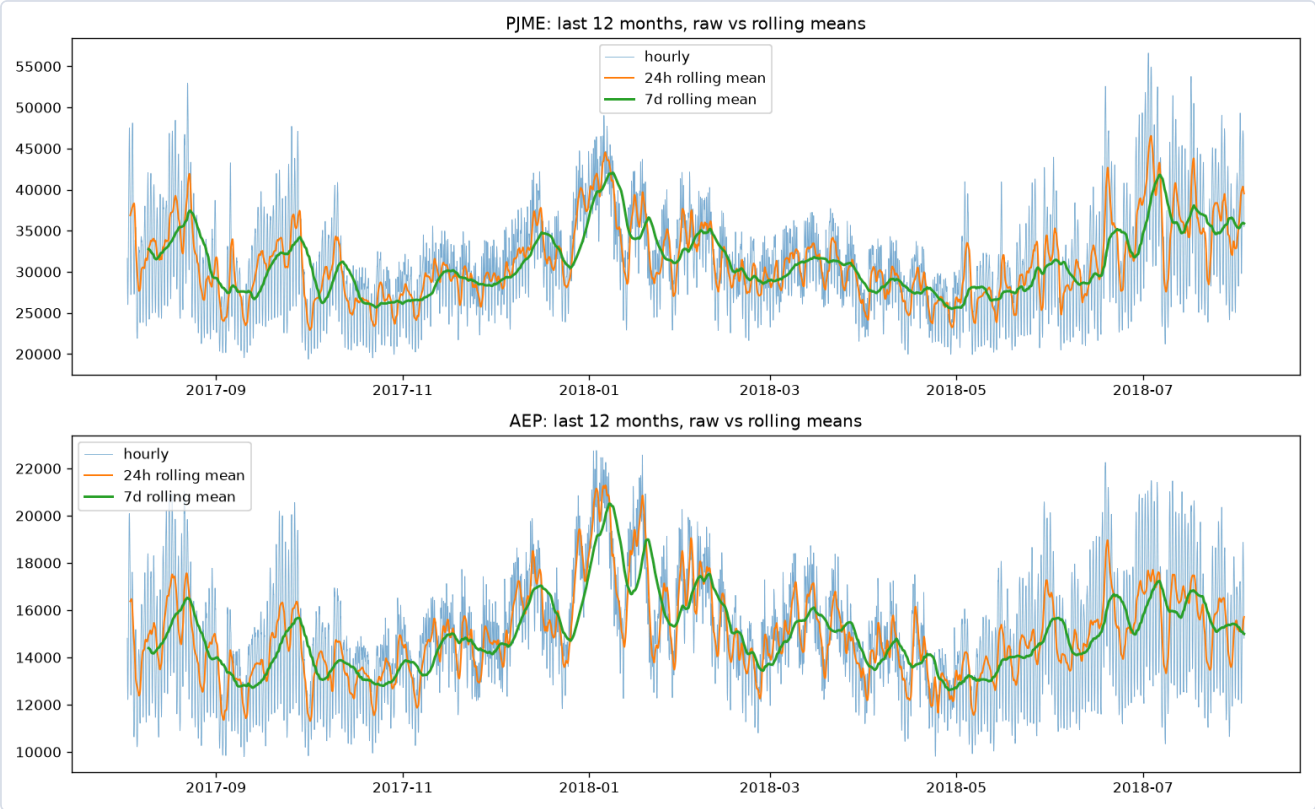


Figure 2.7 — Last 12 months: raw hourly vs 24h and 7-day rolling means, both regions.

The raw hourly series is dominated by the daily on/off cycle, hiding slower structure. The 24h rolling mean removes the intra-day cycle and reveals day-to-day regime shifts (e.g. a heatwave or cold snap); the 7-day rolling mean smooths further and exposes seasonal drift across weeks — motivating `rolling_mean_24h` / `rolling_mean_7d` as features beyond raw lags.

Q6. Do PJME and AEP move together, or does one lead the other?

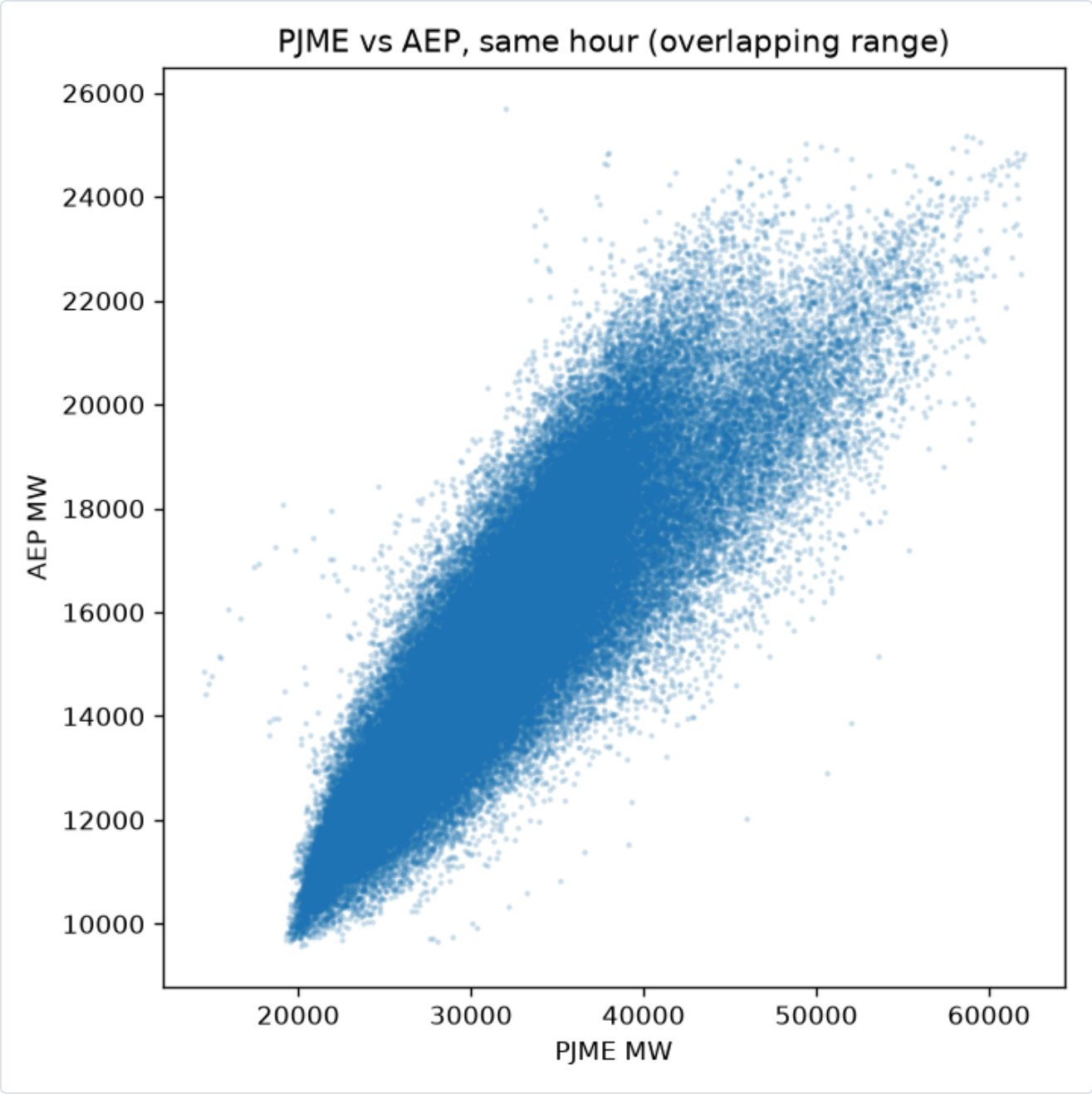


Figure 2.8 — PJME vs AEP, same hour, overlapping date range.

PJME and AEP are strongly positively correlated at the same hour, with no meaningful lead/lag between them — both are PJM-interconnection regions subject to the same broad weather and calendar drivers. This means the two regions don't add much independent predictive signal to each other; each region's own history remains the primary predictor for its own demand.

2.3 Feature Engineering & Preprocessing

Each engineered feature is chosen directly from a finding above:

- **Calendar features** (`hour`, `day_of_week`, `is_weekend`, `is_holiday`, `month`) — from Q2/Q3's seasonal and weekday/holiday effects.
- **Lag features** (`lag_1h`, `lag_24h`, `lag_168h`) — from Q4's ACF/lag correlation.

- **Rolling features** (`rolling_mean_24h`, `rolling_mean_7d`, `rolling_std_24h`) — from Q5's regime-shift finding. Computed on the series shifted 1 hour first, so the current hour's own reading never leaks into its own feature.

The first 168 hours of each region (needed for `rolling_mean_7d`'s full-week lookback) are dropped rather than filled, since fabricating a lag/rolling value with no real history would be misleading. Data is then split **chronologically** — never shuffled — into train / 30-day validation / 30-day test, so validation and test always represent forecasting forward into genuinely unseen future hours.

2.4 Modeling Experiments

Two experiments per region: a Linear Regression baseline (calendar + lag features only), and an XGBoost model (+ rolling features) tuned via `RandomizedSearchCV` over a chronological `TimeSeriesSplit` cross-validation (avoiding look-ahead leakage in the CV itself).

Exp	Region	Model	Features	Key Hyperparameters	Val RMSE	Val MAPE	Test RMSE	Test MAPE
E1	PJME	Linear Regression	calendar + lag	— (baseline)	1545.1	3.63%	1627.4	3.84%
E2	PJME	XGBoost	+ rolling	n_estimators=400, max_depth=5, lr=0.1, subsample=0.7, colsample=0.85	497.3	1.09%	537.7	**1.08%**
E1	AEP	Linear Regression	calendar + lag	— (baseline)	567.4	3.06%	575.6	3.10%
E2	AEP	XGBoost	+ rolling	n_estimators=400, max_depth=5, lr=0.1, subsample=0.7, colsample=0.85	185.7	0.91%	188.5	**0.90%**

> **Result:** XGBoost wins decisively for both regions, cutting test MAPE from 3.84%→1.08% (PJME) and 3.10%→0.90% (AEP) versus the linear baseline. Both winning models are saved to `src/models/artifacts/{region}_winning_model.joblib` and used directly by Task 4's forecast script.

3. Task 2 — Database Design (SQL & MongoDB) Abdalaziz Twariki

3.1 Relational Schema (MySQL)

Normalised to **3NF** across four tables — the wide processed CSV from Task 1 is split so calendar attributes (which depend only on the timestamp, not the region or the reading) live in their own table instead of repeating on every row.

Table	Grain	Purpose
`regions`	one row per PJM sub-region	reference data (PJME, AEP)
`calendar_features`	one row per distinct hour	hour / day-of-week / month / weekend / holiday flags
`energy_readings`	one row per (region, hour)	fact table: mw + lag & rolling features + split label
`predictions`	one row per forecast	model outputs, written by Task 4's script

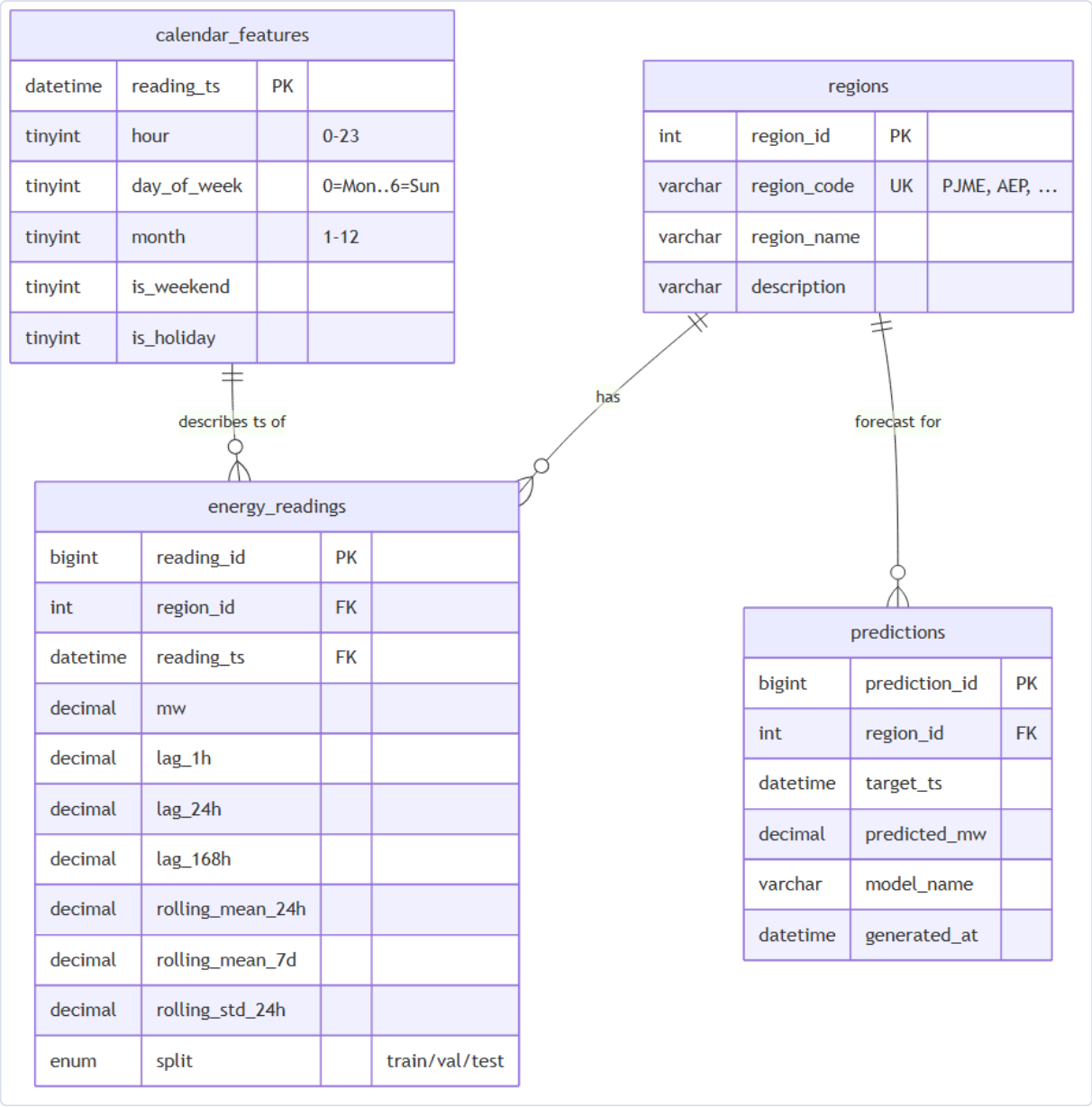


Figure 3.1 — Entity-relationship diagram for the MySQL schema (src/database/sql/schema.sql).

3.2 MongoDB Design

Document-per-region-per-day in the `energy_daily` collection: each document holds one calendar day for one region, with the 24 hourly observations embedded as `hourly_readings[]` and a precomputed `daily_stats` sub-document — a full day's load curve is retrieved in a single read, the natural time-series access pattern, deliberately contrasting with the normalized SQL design. A parallel `predictions` collection mirrors the SQL table.

3.3 Load Results

Store	PJME	AEP
MySQL `energy_readings` rows	145,224	121,128
MongoDB `energy_daily` documents	6,052	5,048

3.4 Queries & Results

5 SQL queries were run (exceeds the ≥3 minimum) — *latest record* and *date-range* are the two required time-series queries, plus three analytical ones:

#	Query	Sample Result
1	Latest record per region *(required)*	PJME: 35,486 MW @ 2018-08-03 00:00; AEP: 14,809 MW @ 2018-08-03 00:00
2	Records by date range *(required)*	PJME, 2015-07-01 → 2015-07-08: 168 hourly rows returned
3	Average load by day-of-week (JOIN)	PJME Mon 32,671 MW → Sun 29,405 MW; AEP Mon 15,813 MW → Sun 14,202 MW
4	7-day rolling average (window function)	PJME daily mean smoothed via `AVG() OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`
5	All-time peak hour per region	PJME: 62,009 MW @ 2006-08-02 17:00; AEP: 25,695 MW @ 2008-10-20 14:00

4 MongoDB queries were run (exceeds the ≥3 minimum):

#	Query	Sample Result
1	Latest document *(required)*	PJME 2018-08-03, avg_mw=35,486.0, peak_hour=0
2	Date range *(required)*	PJME 2015-07-01 → 2015-07-08: 7 day-documents
3	`$group` — monthly average	PJME July avg 37,882 MW (highest); April avg 27,861 MW (lowest)
4	`$unwind` — all-time peak hour	2006-08-02 17:00, 62,009 MW — matches the SQL result exactly, cross-verifying both stores

> Full captured query output (including the complete 168-row date-range result) is committed to the repository at `src/database/query_results.md`.

4. Task 3 — CRUD & Time-Series API Endpoints Nkhomotabo Amazing Mkhonta

Built with FastAPI, using SQLAlchemy for MySQL and PyMongo for MongoDB, with parallel endpoint sets so every operation exists for both databases.

Method	SQL Endpoint	MongoDB Endpoint	Purpose
GET	`/api/sql/readings/latest`	`/api/mongo/readings/latest`	**Required:** latest record
GET	`/api/sql/readings/range`	`/api/mongo/readings/range`	**Required:** date-range query
GET	`/api/sql/readings`	`/api/mongo/readings`	List (paginated)
GET	`/api/sql/readings/{id}`	`/api/mongo/readings/{region}/{date}`	Read one
POST	`/api/sql/readings`	`/api/mongo/readings`	Create
PUT	`/api/sql/readings/{id}`	`/api/mongo/readings/{region}/{date}/{hour}`	Update
DELETE	`/api/sql/readings/{id}`	`/api/mongo/readings/{region}/{date}/{hour}`	Delete
POST	`/api/sql/predictions`	`/api/mongo/predictions`	Store a forecast (used by Task 4)

4.1 Verified Example — Latest Record (Required Endpoint)

`GET /api/sql/readings/latest?region_id=1`


```
{
  "region_id": 1, "reading_ts": "2018-08-03T00:00:00", "mw": 35486.0,
  "lag_1h": 38500.0, "lag_24h": 37158.0, "lag_168h": 35742.0,
  "rolling_mean_24h": 39593.04, "rolling_mean_7d": 35881.5, "rolling_std_24h": 6225.26,
  "split": "test", "reading_id": 145224
}
```


4.2 Full CRUD Cycle — Verified Live

A complete POST → GET → PUT → DELETE cycle was run against the live API for both databases to confirm every operation genuinely works, not just that the routes exist:

Step	SQL Result	MongoDB Result
POST (create)	New row created (`reading_id: 266353`)	Hour 1 appended to the 2018-08-03 day-document (hours `[0, 1]`)
GET (read)	Returns the created row	Document shows the new hour

PUT (update)	`mw` updated 11111 → 22222	`mw` updated 30000 → 31000
DELETE	Row removed; follow-up GET → 404	Hour removed; document restored to original (hours `[0]`)
Duplicate prediction	`409 Conflict`	`409 Conflict`

Both `predictions` stores carry a unique `(region, target_ts, model_name)` key; posting the same forecast twice is rejected with `409 Conflict` and the message *"Prediction already exists for this region, target_ts and model"*, which Task 4's script relies on to skip already-stored forecasts.

> **Design note:** MongoDB's `POST /readings` only appends an hour to an *existing* day-document — it correctly returns `404` if the target region/date has no document yet. Creating a brand-new day from scratch isn't supported via the API; only pre-seeded days (from the Task 2 loader) accept writes. This is a deliberate consequence of the embedded-document design, not an oversight.

Full request/response examples for both databases, including the predictions endpoints, are committed to the repository at `src/api/api_examples.md`.

5. Task 4 — Prediction / Forecast Script Parfait Shema Semana

`src/forecast/run_forecast.py` consolidates all three previous tasks into one script: **fetch** → **preprocess** → **predict** → **write back**, closing the loop end to end.

5.1 Pipeline Steps

1. **Fetch:** calls `GET /api/sql/readings/latest` to find the most recent reading actually in the database, then `GET /api/sql/readings/range` to pull a 14-day trailing window — a single record isn't enough, since `lag_168h` needs a full week of history.
2. **Preprocess:** extends the fetched window with one empty row at the target (next) hour and runs it through Task 1's own `build_feature_set()` (imported directly from `src/preprocessing/features.py`, not reimplemented), so the target's lag/rolling features are computed from real preceding history using the identical logic the model was trained on.
3. **Load model:** loads the region's Task 1 winning model (`src/models/artifacts/{region}_winning_model.joblib` — XGBoost for both regions).
4. **Predict & write back:** predicts the next hour, then POSTs the result to both `/api/sql/predictions` and `/api/mongo/predictions` (Task 3's endpoints).

5.2 Verified Live Run

```
=== Forecasting next hour for AEP ===
Fetching AEP window from /api/sql/readings/range (2018-07-20 01:00 -> 2018-08-03 00:00)
Fetched 336 rows, 2018-07-20 01:00:00 -> 2018-08-03 00:00:00
--- BEFORE: last 3 hours of input for AEP ---
2018-08-02 22:00:00    17001.0
2018-08-02 23:00:00    15964.0
2018-08-03 00:00:00    14809.0
--- AFTER: AEP forecast for 2018-08-03 01:00:00 -> 13660.5 MW ---
SQL prediction stored: {region_id: 2, target_ts: "2018-08-03T01:00:00",
  predicted_mw: 13660.47, model_name: "xgboost", prediction_id: 8}
Mongo prediction stored: {region_code: "AEP", target_ts: "2018-08-03T01:00:00",
  predicted_mw: 13660.47, model_name: "xgboost"}
```

The same run forecast PJME at 32,755.4 MW and stored it in both databases. Both forecasts continue the overnight decline in each region's last few hours (AEP 17,001 → 15,964 → 14,809 MW), the expected sanity check for a next-hour prediction. A design detail worth noting: the script anchors its forecast window on the **latest reading actually in the database** rather than wall-clock time — since this dataset is historical and frozen at 2018-08-03, using `datetime.now()` would fetch an empty window and fail.

> **Robustness, verified:** re-running the script against an already-forecast hour hits the databases' unique `(region, target_ts, model)` key; the API answers `409 Conflict` and the script logs a warning instead of crashing — it still completes successfully and reproduces the same forecast (32,755.4 MW for PJME); only the already-stored write is skipped. The API address is read from `API_BASE_URL` in `.env` and can be overridden with `--api-base`. Full captured output for both regions is in `src/forecast/forecast_example.md`.

6. Team Contributions

Member	Task	Major Components
Hassan Adelani Luqman	Task 1	EDA notebook (dataset understanding, 6 analytical questions, 8 figures); `clean.py` (gap-filling pipeline); `features.py` (calendar/lag/rolling feature engineering, chronological split); `train.py` / `evaluate.py` (Linear Regression + tuned XGBoost experiments, model artifacts); project scaffold (README, requirements, docker-compose)
Abdalaziz Twariki	Task 2	3NF MySQL schema (`schema.sql`) and ERD; `load_mysql.py` loader; MongoDB document-per-day design (`setup.py` , `sample_documents.json`); 5 SQL + 4 MongoDB demonstration queries with captured results (`query_results.md`)
Nkhomotabo Amazing Mkhonta	Task 3	FastAPI application (`main.py` , `schemas.py` , dependency wiring); full CRUD routers for both SQL and MongoDB (`sql_routes.py` , `mongo_routes.py`) including the required latest-record and date-range endpoints; captured API examples (`api_examples.md`)
Parfait Shema Semana	Task 4	`run_forecast.py` : API-anchored window fetch, feature-pipeline reuse, model loading, prediction, and write-back to both databases; captured before/after forecast run (`forecast_example.md`); final integration — `409 Conflict` handling for duplicate predictions across the API and forecast script, pinned dependencies, committed model artifacts, README and report updates

All work was committed directly to `main` in the group repository, with incremental, descriptively-messaged commits per task (e.g. `Add cleaning pipeline for raw hourly PJM series` , `Add normalised MySQL schema and ERD` , `Add MySQL CRUD and time-series endpoints` , `Add forecast script: fetch from API, build features, predict, write back`) rather than a single squashed dump — full history is visible in the repository's commit log.

7. Limitations & Future Work

- **Classical ML only.** Two solid experiments (Linear Regression, tuned XGBoost) were run per region; a deep learning approach (e.g. LSTM) was scoped as an optional stretch goal and not attempted given the project timeline. XGBoost's large margin over the linear baseline (test MAPE ~1% vs ~3.5%) suggests the classical approach already captures most of the exploitable signal.
- **Per-region models.** PJME and AEP are modeled independently rather than jointly; Task 1's Q6 found the two regions move together with no lead/lag, so a joint model was judged unlikely to add much beyond independent per-region models given the time available.
- **Historical, frozen dataset.** The pipeline operates on a fixed 2002–2018 dataset rather than a live feed — Task 4's forecast script had to be deliberately anchored on the latest reading in the database rather than wall-clock time to remain functional. A production deployment would need a live ingestion path.
- **MongoDB write scope.** The MongoDB API's `POST /readings` can only append to an existing day-document, not create a new region/date from scratch — acceptable for this project's scope (all days are pre-seeded by the Task 2 loader) but worth widening if the API needed to support entirely new regions later.
- **Predictions endpoints are write-only.** Both `/predictions` endpoints support POST but not GET/PUT/DELETE — sufficient for Task 4's write-back use case, but a fuller audit trail would benefit from read access too.